torch.nn.functional.embedding_bag#

https://pytorch.org/docs/stable/generated/torch.nn.functional.embedding_bag.

Description:

Compute sums, means or maxes of bags of embeddings. Calculation is done without instantiating the intermediate embeddings. See torch.nn.EmbeddingBag for more details. This operation may produce nondeterministic gradients when given tensors on a CUDA device. See Reproducibility for more information. input (LongTensor) – Tensor containing bags of indices into the embedding matrix

Function Signature

```
torch.nn.functional.embedding_bag(input, weight,
offsets=None, max_norm=None, norm_type=2,
scale_grad_by_freq=False, mode='mean', sparse=False,
per_sample_weights=None, include_last_offset=False,
padding_idx=None)[source]#

Parameters

Return type

Shape:
```

Returns:

Tensor

Code Examples

```
>>> # an Embedding module containing 10 tensors of size 3
>>> embedding_matrix = torch.rand(10, 3)
>>> # a batch of 2 samples of 4 indices each
>>> input = torch.tensor([1, 2, 4, 5, 4, 3, 2, 9])
>>> offsets = torch.tensor([0, 4])
>>> F.embedding_bag(input, embedding_matrix, offsets)
tensor([[ 0.3397,  0.3552,  0.5545],
        [ 0.5893,  0.4386,  0.5882]])

>>> # example with padding_idx
>>> embedding_matrix = torch.rand(10, 3)
>>> input = torch.tensor([2, 2, 2, 2, 4, 3, 2, 9])
>>> offsets = torch.tensor([0, 4])
>>> F.embedding_bag(input, embedding_matrix, offsets,
padding_idx=2, mode='sum')
tensor([[ 0.0000,  0.0000,  0.0000],
        [-0.7082,  3.2145, -2.6251]])
```

Notes & Warnings

NOTE Note This operation may produce nondeterministic gradients when given tensors on a CUDA device. See Reproducibility for more information.

Detailed Documentation

Documentation

Compute sums, means or maxes of bags of embeddings.

Calculation is done without instantiating the intermediate embeddings. See torch.nn.EmbeddingBag for more details.

This operation may produce nondeterministic gradients when given tensors on a CUDA device. See Reproducibility for more information.

input (LongTensor) – Tensor containing bags of indices into the embedding matrix

weight (Tensor) – The embedding matrix with number of rows equal to the maximum possible index + 1, and number of columns equal to the embedding size

offsets (LongTensor, optional) – Only used when input is 1D. offsets determines the starting index position of each bag (sequence) in input.

max_norm (float, optional) – If given, each embedding vector with norm larger than max_norm is renormalized to have norm max_norm. Note: this will modify weight in-place.

norm_type (float, optional) – The p in the p-norm to compute for the max_norm option. Default 2.

scale_grad_by_freq (bool, optional) – if given, this will scale gradients by the inverse of frequency of the words in the mini-batch. Default False. Note: this option is not supported when mode="max".

mode (str, optional) – "sum", "mean" or "max". Specifies the way to reduce the bag. Default: "mean"

sparse (bool, optional) – if True, gradient w.r.t. weight will be a sparse tensor. See Notes under torch.nn.Embedding for more details regarding sparse gradients. Note: this option is not supported when mode="max".

per_sample_weights (Tensor, optional) – a tensor of float / double weights, or None to indicate all weights should be taken to be 1. If specified, per_sample_weights must have exactly the same shape as input and is treated as having the same offsets, if those are not None.

include_last_offset (bool, optional) – if True, the size of offsets is equal to the number of bags + 1. The last element is the size of the input, or the ending index position of the last bag (sequence).

padding_idx (int, optional) – If specified, the entries at padding_idx do not contribute to the gradient; therefore, the embedding vector at padding_idx is not updated during training, i.e. it remains as a fixed “pad”. Note that the embedding vector at padding_idx is excluded from the reduction.

input (LongTensor) and offsets (LongTensor, optional)

If input is 2D of shape (B, N), it will be treated as B bags (sequences) each of fixed length N, and this will return B values aggregated in a way depending on the mode. offsets is ignored and required to be None in this case.

If input is 1D of shape (N), it will be treated as a concatenation of multiple bags (sequences). offsets is required to be a 1D tensor containing the starting index positions of each bag in input. Therefore, for offsets of shape (B), input will be viewed as having B bags. Empty bags (i.e., having 0-length) will have returned vectors filled by zeros.

weight (Tensor): the learnable weights of the module of shape (num_embeddings, embedding_dim)

per_sample_weights (Tensor, optional). Has the same shape as input.

output: aggregated embedding values of shape (B, embedding_dim)
